Library Management System

Backend Architecture & API Reference

For the Frontend Development Team

Version 1.0 · Team of Four

1. System Overview

The Library Management System (LMS) is a Spring Boot microservices application. It exposes a REST API that the frontend consumes. There are four independently deployable services coordinated via Eureka service discovery and a centralised Spring Cloud Config Server.

1.1 Services & Ports

Service	Artifact ID	Port	Responsibility
Config Server	config-server	8888	Centralised configuration for all services
Eureka Server	eureka-server	8761	Service discovery & registry
LMS Core	lms	8081	Books, members, transactions, reservations
Notification Service	notificationservice	8085	Email delivery via SendGrid

1.2 Startup Order

Services must be started in this order for the system to function correctly:

- Config Server (port 8888) — must be healthy first
- Eureka Server (port 8761) — waits for config
- LMS Core (port 8081) — registers with Eureka, fetches config
- Notification Service (port 8085) — registers with Eureka, fetches config

1.3 Base URL

All LMS API endpoints are prefixed with: http://localhost:8081 All Notification endpoints (internal use only): http://localhost:8085

1.4 Design Patterns Used

Pattern	Where Used	Purpose
Strategy	Membership Plans	Swap borrowing rules (Standard / Premium) at runtime
Decorator	Add-ons	Wrap a plan to extend borrowing duration +7 days, +2 books
State	Book Availability	Available → Unavailable → Lost transitions

Command	Admin Book Ops	Undo-able add / remove book operations
Factory	Plan Creation	MembershipPlanFactory validates all PlanType enum values
Circuit Breaker	Transaction Service	Resilience4j wraps borrow/return calls
Retry	Notification REST	3 retries with 2 s backoff before fallback

2. Authentication & Rate Limiting

The current implementation does not include JWT or session-based authentication — member credentials (userName, password) are stored in-memory but no auth token is issued. The frontend should plan for future auth integration.

2.1 Rate Limiting

A custom RateLimitInterceptor guards all /lms/** routes:

Setting	Value
Window	60 seconds
Max requests (dev)	100 per IP
Max requests (prod)	20 per IP
Response on exceed	HTTP 429 + JSON error body

429 Rate Limit Response Body:

```
{
  "error": "Rate limit exceeded. Try again later."
}
```

2.2 CORS

Allowed Origin: http://localhost:5173 (Vite/React dev server) Allowed Methods: GET, POST, PUT, DELETE Paths covered: /lms/** Update CorsConfig.java before deploying to a different frontend origin.

2.3 Global Error Format

All unhandled exceptions return the following JSON envelope with HTTP 500:

Error Response:

```
{
  "message": "Human-readable error description",
  "status": 500,
  "timestamp": "2025-04-21T14:32:00"
}
```

Business rule violations (e.g. borrowing limit exceeded) throw IllegalArgumentException or ServerException, which are caught by GlobalExceptionHandler and returned in this same format.

3. Books API

Manage the library book catalogue. All routes sit under /lms/. Add / Remove operations use the Command pattern, enabling undo.

GET **/lms/getAllBooks** — List all books in the catalogue

No request body or query parameters needed.

Response 200 — Array of BookDTO:

```
[
  {
    "id": 493827364819201,
    "isbn": "978-0132350884",
    "title": "Clean Code",
    "author": "Robert C. Martin",
    "category": "Technology",
    "publicationYear": 2008,
    "copiesAvailable": 4,
    "available": true
  },
  ...
]
```

POST **/lms/addBook** — Add a new book (stored in command history for undo)

Request Body (AddBookRequest):

```
{
  "isbn": "978-0132350884",        // required — string
  "title": "Clean Code",           // required — string
  "author": "Robert C. Martin",    // required — string
  "category": "Technology",        // required — string
  "publicationYear": 2008,         // required — integer
  "copiesAvailable": 5             // required — integer >= 1
}
```

Response 200 — Plain string:

```
"Book added with Id: 493827364819201"
```

POST **/lms/removeBook/{bookId}** — Soft-delete a book (marks copies = 0, state = Lost)

Path parameter: bookId (Long). The book is NOT deleted from storage; it is marked unavailable so undo can restore it.

Response 200:

```
"Book marked as unavailable: Clean Code"
```

Response 500 (book not found):

```
{
  "message": "Book not found with ID: 493827364819201",
```

```
  "status": 500,
  "timestamp": "2025-04-21T14:32:00"
}
```

`POST` `/lms/undoLastBookOperation` — Undo the most recent add or remove command

Pops the top command off the history stack and reverses it. If the last command was addBook, the book is marked unavailable. If it was removeBook, the original copy count is restored.

Response 200:

```
"Book addition undone: ID 493827364819201 marked as unavailable"
// or
"Book 'Clean Code' restored with 5 copies available"
```

Response 200 (nothing to undo):

```
"Cannot undo: No commands to undo"
```

`GET` `/lms/commandHistory` — View the undo history stack

Response 200:

```
{
  "historySize": 2,
  "canUndo": true,
  "lastCommand": "Add Book: 'Clean Code' by Robert C. Martin (ISBN: 978-
0132350884)",
  "allCommands": [
    "Remove Book: ID 112233 (Old Book)",
    "Add Book: 'Clean Code' by Robert C. Martin (ISBN: 978-0132350884)"
  ]
}
```

Response 200 (empty history):

```
{
  "message": "No commands in history",
  "historySize": 0,
  "canUndo": false
}
```

`DELETE` `/lms/commandHistory` — Clear the undo history

Response 200:

```
"Command history cleared successfully"
```

4. Members API

Register library members and query member details. Plans and add-ons determine borrowing limits, fine rates, and reservation caps.

4.1 Membership Plans

Plan	Cost/mo	Borrow Limit	Duration	Max Reservations
STANDARD	$5	10 books	30 days	2
PREMIUM	$7	10 books	30 days	5

> Fine Rate: $0.25 per overdue day for both plans.

4.2 Add-Ons

AddOn Enum	Extra Cost	Borrow Limit +	Duration +
EXTENDED_BORROWING	+$3/mo	+2 books	+7 days

`POST` `/lms/registerMember` — Register a new member

Request Body (RegisterMemberRequest):

```
{
  "userName": "johndoe",          // required
  "password": "secret123",        // required
  "emailId": "john@example.com",  // required — used for notifications
  "phoneNumber": "0851234567",    // required
  "planType": "STANDARD",         // required — STANDARD | PREMIUM
  "memberShipMonths": 12,         // required — integer, months of membership
  "addOns": "EXTENDED_BORROWING"  // optional — omit if no add-on wanted
}
```

Response 200 — Plain string:

```
"Member created with Id : 827364918203744
Selected Plan of Member Standard Plan
Total Cost of the Plan 5"
```

If EXTENDED_BORROWING add-on is included, the plan name remains 'Standard Plan' / 'Premium Plan' but cost and limits are adjusted by the decorator.

Response 500 (invalid plan type):

```
{
```

```
  "message": "Plan not found for type: INVALID. Available types: [STANDARD,
PREMIUM]",
  "status": 500,
  "timestamp": "2025-04-21T14:32:00"
}
```

GET **/lms/getAllMembers** — Retrieve all registered members

Response 200 — Array of MemberDTO:

```
[
  {
    "id": 827364918203744,
    "userName": "johndoe",
    "emailId": "john@example.com",
    "phoneNumber": "0851234567",
    "membershipStartDate": "2025-04-21",
    "membershipEndDate": "2026-04-21",
    "planName": "Standard Plan",
    "activeBorrowCount": 1,
    "activeReservationCount": 0,
    "queuedReservationCount": 1
  }
]
```

5. Transactions API

Handle book borrowing and returns. Both operations trigger an email notification via the Notification Service. A Resilience4j Circuit Breaker (name: bookService) wraps both endpoints — if it opens, a fallback message is returned instead of an error.

5.1 Circuit Breaker States

State	Frontend Behaviour
CLOSED	Normal — requests succeed
OPEN	Returns fallback message (HTTP 200 but with warning text — check message content)
HALF_OPEN	Trial requests — may succeed or trigger fallback

> Frontend tip: Check if the success response string contains 'temporarily unavailable' to detect an open circuit breaker.

`GET /lms/borrowBook/?book_id={bookId}&member_id={memberId}` — Borrow a book

Note: Uses GET with query parameters (not REST-ideal but matches the current implementation).

Query Param	Type	Description
book_id	Long	ID of the book to borrow
member_id	Long	ID of the borrowing member

Response 200 (success):

```
"Book borrowed successfully. Transaction ID: 748291038465712
Please note this ID — it is required when returning the book."
```

Response 500 — Book unavailable or has reservations:

```
{
  "message": "Book 'Clean Code' is not available. Please reserve the book to join
the reservation queue.",
  "status": 500,
  "timestamp": "2025-04-21T14:32:00"
}
```

Response 500 — Borrowing limit exceeded:

```
{
  "message": "Borrowing limit exceeded. Current: 10, Limit: 10",
```

```
  "status": 500,
  "timestamp": "2025-04-21T14:32:00"
}
```

Response 200 — Circuit breaker open (fallback):

```
"The borrowing service is temporarily unavailable. Please try again in a few
moments. (Circuit breaker is open — too many recent failures detected.)"
```

`GET` `/lms/processReturn/{transactionId}` — Return a borrowed book

Path parameter: transactionId (Long) — the ID returned when the book was borrowed.

Response 200 (on-time return):

```
"Book 'Clean Code' returned successfully ON TIME. Thank you for returning by the
due date (2025-05-21T10:30:00)!"
```

Response 200 (late return):

```
"Book 'Clean Code' returned LATE. Overdue fine: $2.50 (10 days late). Due date was:
2025-05-21T10:30:00. Returned on: 2025-05-31T09:00:00"
```

Response 500 — Transaction not found:

```
{
  "message": "Transaction not found with ID: 748291038465712",
  "status": 500,
  "timestamp": "2025-04-21T14:32:00"
}
```

Response 500 — Already returned:

```
{
  "message": "Transaction #748291038465712 is already closed. Status: RETURNED",
  "status": 500,
  "timestamp": "2025-04-21T14:32:00"
}
```

`GET` `/lms/getAllTransactions` — List all transactions

Response 200 — Array of TransactionDTO:

```
[
  {
    "id": 748291038465712,
    "bookId": 493827364819201,
    "bookTitle": "Clean Code",
    "bookIsbn": "978-0132350884",
    "memberId": 827364918203744,
    "memberUserName": "johndoe",
    "borrowDate": "2025-04-21T10:30:00",
    "returnedDate": null,             // null if still active
    "dueDate": "2025-05-21T10:30:00",
    "transactionStatus": "ACTIVE",    // ACTIVE | RETURNED | FINE_PENDING
    "overdue": false,
    "daysOverdue": 0
  }
]
```

6. Reservations API

Members can reserve books that are currently unavailable. Reservations follow a FIFO queue. When a book becomes available (after a return), the first QUEUED reservation is auto-promoted to ACTIVE and the member receives an email.

6.1 Reservation Lifecycle

Status	Description
QUEUED	Created — waiting for the book to become available
ACTIVE	Book is available — member has a window to pick it up (2 days)
FULFILLED	Member has picked up the book (processReservationPickup called)
EXPIRED	Member did not pick up within 2 days — reservation lapsed

> Reservation validity: 2 days from the moment status changes to ACTIVE. Standard plan max reservations: 2 active+queued combined. Premium plan max reservations: 5 active+queued combined.

`POST /lms/reserveBook?book_id={bookId}&member_id={memberId}` — Reserve an unavailable book

A reservation can only be created when the book is NOT currently available. If the book is available the API returns an error directing the user to borrow directly.

Query Param	Type	Description
book_id	Long	ID of the book to reserve
member_id	Long	ID of the member making the reservation

Response 200:

```
"Reservation created with id:928374610293847
Your reservation queue number is:2"
```

Response 500 — Book is currently available:

```
{
  "message": "Book 'Clean Code' is currently available. Please borrow it directly
instead of reserving.",
  "status": 500,
  "timestamp": "2025-04-21T14:32:00"
}
```

Response 500 — Duplicate reservation:

```
{
  "message": "You already have an active reservation for 'Clean Code'",
  "status": 500,
  "timestamp": "2025-04-21T14:32:00"
}
```

Response 500 — Limit exceeded:

```
{
  "message": "Reservation limit exceeded. Current active reservations: 2, Limit:
2",
  "status": 500,
  "timestamp": "2025-04-21T14:32:00"
}
```

`POST` `/lms/processReservationPickup/{reservationId}` — Confirm a reservation pickup

Called when the member physically collects the book. Sets reservation status to FULFILLED.

Response 200:

```
"Reservation fulfilled. Book borrowing successfull."
```

Response 500 — Reservation not found:

```
{
  "message": "Reservation not found with ID: 928374610293847",
  "status": 500,
  "timestamp": "2025-04-21T14:32:00"
}
```

`POST` `/lms/expireActiveReservations` — Manually trigger expiry check for all active reservations

Intended for admin/cron use. Finds all ACTIVE reservations past their 2-day window and marks them EXPIRED. Returns HTTP 200 with no body.

`GET` `/lms/getAllReservations` — List all reservations

Response 200 — Array of ReservationDTO:

```
[
  {
    "id": 928374610293847,
    "bookId": 493827364819201,
    "bookTitle": "Clean Code",
    "bookIsbn": "978-0132350884",
    "memberId": 827364918203744,
    "memberUserName": "johndoe",
    "reservationDate": "2025-04-21",
    "expiryDate": "2025-04-23",       // null if still QUEUED
    "status": "ACTIVE",               // QUEUED | ACTIVE | FULFILLED | EXPIRED
    "daysUntilExpiry": 2              // 0 if expired or queued
  }
]
```

7. Resilience & Circuit Breaker

The LMS exposes circuit breaker health for monitoring dashboards.

`GET /lms/circuitBreakers` — Current state of all circuit breakers

Response 200:

```
{
  "bookService": {
    "state": "CLOSED",
    "failureRate": "-1.0%",
    "numberOfBufferedCalls": 3,
    "numberOfFailedCalls": 0,
    "numberOfSuccessfulCalls": 3,
    "description": "Normal operation — requests passing through"
  },
  "memberService": {
    "state": "CLOSED",
    "failureRate": "-1.0%",
    "numberOfBufferedCalls": 1,
    "numberOfFailedCalls": 0,
    "numberOfSuccessfulCalls": 1,
    "description": "Normal operation — requests passing through"
  }
}
```

Response 200 (no calls made yet):

```
{
  "message": "No circuit breakers have been invoked yet. Call /lms/borrowBook/ or
/lms/processReturn/ to activate them."
}
```

7.1 Circuit Breaker Config

Setting	Dev Value	Prod Value
Failure rate threshold	80%	50%
Sliding window size	5 calls	10 calls
Wait in open state	5 s	30 s
Half-open trial calls	2	2
Notification retry attempts	3	3
Notification retry wait	2 s	2 s

8. Notification Service

The Notification Service (port 8085) is called internally by LMS Core — the frontend does not call it directly. It sends transactional emails via SendGrid.

8.1 Notification Triggers

Event	Trigger Point	Email Subject
BORROWED	Successful borrowBook call	Book Borrowed Successfully
RETURNED	On-time processReturn call	Book Returned
OVERDUE	Late processReturn call	Overdue Notice - Fine Applied
RESERVATION_CREATED	Successful reserveBook call	Reservation Confirmed
RESERVATION_AVAILABLE	Auto on book return — next in queue	Your Reserved Book is Available!

8.2 Failure Handling

Notification failures are non-blocking. If the Notification Service is unreachable:

- Resilience4j retries up to 3 times with 2 s between attempts
- If all retries fail, publishFallback logs a warning — the borrow/return operation still succeeds
- The frontend receives the normal success response regardless

Frontend note: Email delivery failure is silent from the API's perspective. A success response does NOT guarantee the email was sent.

9. Complete Data Models

9.1 BookDTO

Field	Type	Nullable	Notes
id	Long	No	Random 15-digit ID generated on creation
isbn	String	No	Book ISBN string
title	String	No	Book title
author	String	No	Author name
category	String	No	Genre / category
publicationYear	Integer	No	Year published
copiesAvailable	Integer	No	Current available copies
available	Boolean	No	true if state == Available

9.2 MemberDTO

Field	Type	Nullable	Notes
id	Long	No	Random 15-digit ID
userName	String	No	Login username
emailId	String	No	Email for notifications
phoneNumber	String	No	Contact number
membershipStartDate	LocalDate	No	yyyy-MM-dd
membershipEndDate	LocalDate	No	Start + memberShipMonths
planName	String	No	Standard Plan / Premium Plan
activeBorrowCount	Integer	No	Currently borrowed books
activeReservationCount	Integer	No	ACTIVE reservations
queuedReservationCount	Integer	No	QUEUED reservations

9.3 TransactionDTO

Field	Type	Nullable	Notes
id	Long	No	Transaction ID (needed for return)

bookId	Long	No	
bookTitle	String	No	
bookIsbn	String	No	
memberId	Long	No	
memberUserName	String	No	
borrowDate	LocalDateTime	No	ISO-8601 datetime string
returnedDate	LocalDateTime	Yes	null while active
dueDate	LocalDateTime	No	borrowDate + plan duration
transactionStatus	String Enum	No	ACTIVE | RETURNED | FINE_PENDING
overdue	Boolean	No	true if dueDate is past
daysOverdue	Long	No	0 if not overdue

9.4 ReservationDTO

Field	Type	Nullable	Notes
id	Long	No	Reservation ID (needed for pickup)
bookId	Long	No	
bookTitle	String	No	
bookIsbn	String	No	
memberId	Long	No	
memberUserName	String	No	
reservationDate	LocalDate	No	Date created (yyyy-MM-dd)
expiryDate	LocalDate	Yes	null until status = ACTIVE
status	String Enum	No	QUEUED | ACTIVE | FULFILLED | EXPIRED
daysUntilExpiry	Long	No	0 when QUEUED or expired

10. Enum Reference

PlanType

- STANDARD
- PREMIUM

AddOns

- EXTENDED_BORROWING — adds +2 borrow limit, +7 days duration, +$3/mo cost

TransactionStatus

- ACTIVE — book is currently borrowed
- RETURNED — book has been returned, no fine
- FINE_PENDING — returned late, fine calculated

ReservationStatus

- QUEUED — waiting for the book to become available
- ACTIVE — book available, member has 2 days to collect
- FULFILLED — member picked up the book
- EXPIRED — 2-day pickup window lapsed

NotificationType

- BORROWED, RETURNED, OVERDUE, RESERVATION_CREATED, RESERVATION_AVAILABLE

11. API Quick Reference

Method	Endpoint	Description
GET	/lms/getAllBooks	List all books
POST	/lms/addBook	Add a book (body: AddBookRequest)
POST	/lms/removeBook/{bookId}	Soft-delete a book
POST	/lms/undoLastBookOperation	Undo last book add/remove
GET	/lms/commandHistory	View undo stack
DELETE	/lms/commandHistory	Clear undo stack
POST	/lms/registerMember	Register a member (body: RegisterMemberRequest)
GET	/lms/getAllMembers	List all members
GET	/lms/borrowBook/	Borrow a book (?book_id=&member_id=)
GET	/lms/processReturn/{txId}	Return a book
GET	/lms/getAllTransactions	List all transactions
POST	/lms/reserveBook	Reserve a book (?book_id=&member_id=)
POST	/lms/processReservationPickup/{id}	Confirm reservation pickup
POST	/lms/expireActiveReservations	Expire overdue reservations (admin/cron)
GET	/lms/getAllReservations	List all reservations
GET	/lms/circuitBreakers	Circuit breaker health status

Known Frontend Considerations

- IDs are large random Longs (up to 15 digits) — use BigInt or string handling in JavaScript
- All success responses from borrow/return/reserve are plain strings, not JSON objects
- Error responses always follow the ErrorResponse JSON format { message, status, timestamp }
- The Transaction ID returned by borrowBook must be stored locally to process returns
- No pagination — all list endpoints return the full collection (in-memory store, dev only)
- There is no authentication endpoint yet — plan for future JWT integration
- Borrow and return operations are GET requests (not POST) per current controller mapping